Homework 2

Problem 1: PSOLA pitch and duration modification (40 points)

When you listen to someone speak, the pitch of the voice and the speed of the words seem like two separate things, and the formants you studied in Homework 1, which give each vowel its identity, seem like a third. In this problem you will see how far apart they really are using Praat's implementation of PSOLA through Parselmouth, the library you used in Homework 1.

The zip file *hw2_data.zip* on Canvas contains a folder *speech_clips/* with six 16 kHz recordings and their transcripts (*references.csv*), a text file *my_script.txt* that you will read aloud in part (c), and *sentences.csv*, which you will need in Problem 3. In parts (a,b) you will work with the two clean recordings, *clean_lj* and *clean_jfk* (F0 range: 75–500 Hz; formant ceiling: 5,500 Hz for *clean_lj*, 5,000 Hz for *clean_jfk*). Throughout the homework, a "ratio" means the value after a change divided by the value before it, and F0, F1 and F2 mean the medians over voiced frames, measured as in Homework 1.

(a) Apply PSOLA (12 points)

Begin by creating five modified versions of each clip with PSOLA

- one with the pitch raised by a factor of 1.5,
- one with it lowered by a factor of 0.75,
- one made 0.8 times as long (faster),
- one made 1.5 times as long (slower), and
- one in which the pitch contour is replaced by a constant equal to the clip's median F0, which turns the speaker into a monotone.

Save every version as a WAV file and listen to it. Some of these will sound natural and others will not, and it is worth forming your own opinion about which is which before you look at any numbers.

Then measure what you did. In your report, include a table with one row for each clip and version (ten rows) giving the duration ratio and the F0 ratio; for the two monotone versions also give the standard deviation of F0 before and after, since that is the quantity you actually changed. Plot the F0 contours of the original, the pitch × 0.75 and the pitch × 1.5 versions of *clean_lj* on a single axis. Finish by discussing your results and whether your numbers support the claim that PSOLA controls pitch and duration independently of each other.

(b) PSOLA vs. resampling (12 points)

PSOLA is not the only way to change the pitch of a recording. The simplest way is something those of us old enough may have done by speeding up a tape: play the samples back faster and everything goes up in pitch, play them slower and everything goes down. In software this is done by resampling. Create pitch × 0.75 and × 1.5 versions of both clips this way, by resampling the signal and then treating the result as a 16 kHz recording again, so that the sample rate stored in the file does not change. Listen to these next to the PSOLA versions from part (a).

For both methods, both clips and both factors, report the ratios of duration, F0, F1 and F2 in a single table, and plot spectrograms of the first 3 s of *clean_lj* side by side for the original, the PSOLA × 1.5 version and the resampled × 1.5 version. Then discuss which method preserves the formants and the duration and why. In Homework 1 you saw that speakers with different vocal-tract lengths have formants at different frequencies; use that to say which of the two methods would make a listener think they were hearing a different speaker.

(c) Your own voice (16 points)

Measurements on course recordings are convincing but hearing PSOLA work on your own voice is way better. Read the passage in *my_script.txt* aloud in a quiet room (it takes about 20 seconds) and save it as *my_voice.wav*. Then record it a second time with some background noise, for example a fan, music or a video playing, and save it as *my_voice_noisy.wav*. Any recorder will do (Audacity, a phone app or Python's *sounddevice* package) as long as you export a WAV file. You will use both recordings again in Problems 2 and 3, so keep them.

Choose analysis settings that suit your voice: a pitch range of about 75–300 Hz if your voice is lower, or 100–500 Hz if it is higher, and a formant ceiling of 5,000 or 5,500 Hz. Using *my_voice.wav*, create four versions:

- PSOLA with the pitch × 0.75,
- PSOLA with the pitch × 1.5,
- a PSOLA monotone, and
- a resampled version with the pitch × 1.5.

Listen to all four. In your report, give your own median F0, F0 standard deviation, F1 and F2, and a table of the duration, F0, F1 and F2 ratios for the four versions, as in part (b). Then discuss which versions still sound like you and which sound artificial, and connect what you hear to the numbers.

Problem 2: Forced alignment with the Montreal Forced Aligner (25 points)

Many questions about speech are questions about timing: how long a vowel lasts, where in a word a formant should be measured, how fast someone talks, where one word ends and the next begins. Answering them by hand in Praat means placing boundaries on a TextGrid one at a time, which is slow and does not scale. A forced aligner automates this: you give it a recording and its transcript, and it finds the most probable time for every word and phone boundary. The Montreal Forced Aligner (MFA) does this with a pretrained English acoustic model (*english_us_arpa*) and a pronunciation dictionary, and it writes the result as a Praat TextGrid with a *words* tier and a *phones* tier. The phones are ARPAbet symbols, and a vowel whose label ends in 1 carries primary stress (for example *AY1*).

To prepare, make one folder in which each utterance is a pair of files: *name.wav* and *name.lab*, a plain-text file containing the transcript. Use five utterances:

- *my_voice*,
- *my_voice_noisy*,
- *my_voice_slow* (your *my_voice* made 1.5 times longer with PSOLA), and
- the course clip *clean_lj*, whose transcript is in *references.csv*
- the course clip *clean_jfk*, whose transcript is also in *references.csv*.

Run MFA once on the whole folder and read the five TextGrids back into Python.

(a) Align and inspect (8 points)

Before using an alignment, look at it, and the right tool for looking is Praat itself. MFA writes its TextGrids to the output folder you gave it (one *.TextGrid* per utterance, with the same base name as the recording). In Praat, open *my_voice.wav* (Open > Read from file) and its TextGrid, select both objects in the Praat object list, and click View & Edit. You will see the waveform, the spectrogram (turn it on in the Spectrum menu if it is hidden) and the two tiers, *words* and *phones*. Zoom in and click on intervals to hear them. Look especially at places where a boundary is visible in the spectrogram, such as a stop burst or the onset of an *s*.

In your report, include a screenshot of the Praat editor showing about three seconds of *my_voice* with both tiers and the spectrogram. Point out one word or phone boundary that is placed very well and one that you think is wrong or arguable and explain what you see (or hear) that tells you so. Then open the TextGrid of *my_voice_noisy* in the same way and say whether the boundaries moved noticeably.

To put numbers on that comparison, read the TextGrids into Python as well. Report a table with one row for each of the five utterances giving the number of words, the number of phones, the time of the first word start, the time of the last word end, and the number of words the aligner could not find in its dictionary (they appear as *<unk>* or *spn*). Because *my_voice* and *my_voice_noisy* contain the same words, you can compare their word start times directly: report the mean and maximum difference in milliseconds and name the word on which the two alignments disagree most. Discuss how the noise affected the alignment, and whether the numbers agree with what you saw in Praat.

(b) Speaking rate and vowels (10 points)

Now use the alignments to measure something. First, speaking rate: for *my_voice*, *clean_lj* and *clean_jfk*, report the number of phones per second between the start of the first word and the end of the last word, the fraction of that time spent in pauses, and the mean phone duration. Second, vowels. As in Homework 1, measure F1 and F2 with the Burg method, but this time let the alignment tell you where to look: for every stressed vowel (a phone label ending in 1) take the median F1 and F2 over the middle half of the vowel. For each speaker, report the median F1, F2 and duration of every vowel that occurs at least three times, and plot these medians in an F2–F1 chart with both axes reversed, as you did in Homework 1.

Compare yourself with the two course speakers, citing your numbers, and say how far you would trust a median computed from so few vowel tokens.

(c) Alignment of slowed speech (7 points)

The last question connects this problem to the previous one. Your *my_voice_slow* recording is the same speech as *my_voice*, stretched by PSOLA to 1.5 times its length, so a good aligner should find every word 1.5 times as long. Divide the duration of each word in *my_voice_slow* by its duration in *my_voice*, and report the median, minimum and maximum of these ratios along with the four words with the most extreme ratios. Discuss what ratio you expected, whether the aligner agrees, and what a deviation would tell you: is it more likely a flaw in PSOLA or in the alignment?

Problem 3: Text-to-speech and speech recognition (35 points)

Modern speech technology is built from two halves that mirror each other: text-to-speech (TTS) turns words into a waveform, and automatic speech recognition (ASR) turns a waveform back into words. In this problem you will connect them. A TTS model will read a set of sentences aloud, an ASR model will listen to the result, and you will study where the round trip goes wrong. You will then ask how well the same recognizer copes with real speech, with noisy speech, with PSOLA-modified speech, and with your own voice. You will use the text-to-speech model *facebook/mms-tts-eng* (a single English voice) and the recognizers *openai/whisper-tiny* and *openai/whisper-base*, all from Hugging Face. The first run downloads about 0.5 GB of model files, and a laptop CPU is sufficient. Throughout, recognition quality is measured by the word error rate (WER), computed with the *jiwer* package after converting both texts to lower case and removing punctuation.

The file *sentences.csv* contains 12 sentences of six types: plain sentences, sentences with numbers, with abbreviations, with names, with homographs (words spelled alike but pronounced

differently) and long sentences. The column *text* is what you give to the synthesizer, and the column *spoken* is what a person reading it aloud would actually say. The *spoken* column is the reference for computing WER.

(a) Text to speech (10 points)

Synthesize all 12 sentences with the TTS model, save the WAV files, and listen to every one of them. In your report, give a table with each sentence's audio duration, the time the model took to synthesize it, and the real-time factor (synthesis time divided by audio duration, so that values below 1 mean faster than real time). Include in the table the text that the model's tokenizer actually kept for each sentence, which you can print from the tokenizer. Describe the problems you hear, citing sentence numbers and types, and say how the tokenizer output explains at least one of them.

(b) Speech recognition (10 points)

Now let the recognizers listen. Transcribe the 12 synthetic sentences and the six recordings in *speech_clips/* with both Whisper models, using greedy decoding and telling the model that the language is English, and compute the WER of every transcript against its reference. Report the mean WER by sentence type for the synthetic speech, and a summary table with one row per model and columns for synthetic speech, real clean speech and real noisy speech. Quote the synthetic sentence with the highest WER for *whisper-base* both as it was said and as it was heard.

Discuss which sentence types are hardest and what kind of "error" many of them turn out to be, how whisper-tiny compares with whisper-base, and how recognition of synthetic speech compares with recognition of real clean and real noisy speech.

(c) Does ASR survive PSOLA? (8 points)

In Problem 1 you saw that PSOLA and resampling change a recording in different ways. Here you will see whether a recognizer cares. Take the three clean course recordings and create versions with PSOLA pitch factors of 0.6, 0.8, 1.0, 1.25, 1.6 and 2.0, versions with resampled pitch at the same factors, and versions with PSOLA duration factors of 0.5, 0.75, 1.0, 1.5 and 2.0. Transcribe every version with *whisper-base* and plot the mean WER against the factor, with the PSOLA and resampling pitch curves on one axis (pitch factors on a logarithmic scale) and the duration curve on its own.

Discuss which kind of change hurts recognition most, whether PSOLA or resampling is more harmful at the same pitch factor, and how this connects to what you found in Problem 1(b).

(d) Your own voice (7 points)

Finally, test the recognizers on yourself. Transcribe *my_voice* and *my_voice_noisy* with both Whisper models, and also synthesize the text of *my_script.txt* with the TTS model and transcribe that. Report the WER against the script for all three kinds of speech and both models, and show the whisper-base transcript of each. Discuss which words were recognized wrongly and why, how much the noise hurt, and whether it was easier for Whisper to recognize you or the synthetic voice.

Hints

PSOLA with Parselmouth (Problem 1).

- *snd = parselmouth.Sound(path)*; *m = call(snd, "To Manipulation", 0.01, 75, 500)* (time step, F0 floor, F0 ceiling), with *from parselmouth.praat import call*.
- Pitch: *pt = call(m, "Extract pitch tier")*; *call(pt, "Multiply frequencies", snd.xmin, snd.xmax, 1.5)*; *call([pt, m], "Replace pitch tier")*. Monotone: build a tier with *call("Create PitchTier", "flat", snd.xmin, snd.xmax)* and one point added with *call(flat, "Add point", snd.xmin, f0)*, then replace the pitch tier with it.
- Duration: *dt = call(m, "Extract duration tier")*; *call(dt, "Add point", snd.xmin + 0.001, 1.5)* (1.5 = 50% slower); *call([dt, m], "Replace duration tier")*.
- Resynthesis: *new = call(m, "Get resynthesis (overlap-add)")*; save with *new.save("name.wav", "WAV")*.
- Measuring: *snd.to_pitch(0.01, 75, 500).selected_array["frequency"]* (0 = unvoiced) and *snd.to_formant_burg(0.01, 5, 5500, 0.025, 50)*; take medians over voiced frames. Widen the F0 range when you measure a pitch-shifted sound (for example floor × 0.7 and ceiling × 1.5 × factor), or the tracker will lose the pitch.
- Resampling: *scipy.signal.resample_poly(x, 1000, int(1000 * factor))* raises pitch and formants by *factor* and shortens the sound by the same factor. Wrap a NumPy array back into a sound with *parselmouth.Sound(x, sampling_frequency=16000)*.
- Recording: *sd.rec(20 * 16000, samplerate=16000, channels=1)*, then *sd.wait()* and *soundfile.write("my_voice.wav", x, 16000)* (*import sounddevice as sd*). Stay 20–30 cm from the microphone. Files from other recorders may be 44.1/48 kHz or stereo: read them with *soundfile*, average the channels and resample to 16 kHz before analysis.

Forced alignment (Problem 2).

- Install once: install Miniconda, then in an Anaconda Prompt (Windows) or terminal: *conda create -n aligner -c conda-forge montreal-forced-aligner*, *conda activate aligner*, *pip install praat-parselmouth soundfile pandas scipy matplotlib notebook*. Start Jupyter from this environment. Keep Problem 2 in its own notebook so that this environment does not need the Whisper and TTS packages.
- Models: *mfa model download acoustic english_us_arpa* and *mfa model download dictionary english_us_arpa*. Align: *mfa align --clean --single_speaker CORPUS_DIR english_us_arpa english_us_arpa OUTPUT_DIR*. From Python use *subprocess.run([...], capture_output=True, text=True)*; print its *stdout* and *stderr* and check *returncode*, otherwise MFA's error message appears only in the Jupyter terminal. The transcript file is plain text with the same base name (*my_voice.lab*).
- Reading a TextGrid: *tg = parselmouth.read(path)*; *call(tg, "Get number of intervals", tier)*; *call(tg, "Get start time of interval", tier, i)*, *"Get end time of interval"*, *"Get label of interval"* (tiers and intervals count from 1; empty labels are silence). The *textgrid* or *praatio* packages also work.

TTS and ASR (Problem 3).

- TTS: *tok = AutoTokenizer.from_pretrained("facebook/mms-tts-eng")* and *model = VitsModel.from_pretrained("facebook/mms-tts-eng")*; *ids = tok(text, return_tensors="pt")*; *model(**ids).waveform[0]* is a 16 kHz signal. Call *torch.manual_seed(0)* before each synthesis (VITS is random), and *tok.decode(ids["input_ids"][0])* shows what the model kept.
- Whisper: *WhisperProcessor* and *WhisperForConditionalGeneration* from *openai/whisper-tiny* or *openai/whisper-base*; *processor(samples, sampling_rate=16000, return_tensors="pt").input_features* →

model.generate(features, language="en", task="transcribe") → *processor.batch_decode(ids, skip_special_tokens=True)*. The *pipeline("automatic-speech-recognition", model=...)* helper is a shorter alternative.
- *jiwer.wer(reference, hypothesis)* works on strings; *jiwer.process_words(...)* also gives substitutions, deletions and insertions. Timing: *time.perf_counter()*.

Deliverables

- A code archive (.zip or a link to a Git repository) containing all scripts/notebooks needed to reproduce every number and plot in Problems 1–3, plus a short README describing how to run it. Include your recordings (*my_voice.wav*, *my_voice_noisy.wav*). Jupyter notebooks are welcome.
- A single report (PDF) containing every requested plot, table, and reported value from Problems 1–3, and a discussion of what they show. Also submit the WAV files from Problems 1(a) and 1(c).
- Do not submit the dataset or model checkpoints, only your code, audio you created and report.

Academic Integrity

This is individual work. You may discuss general concepts with classmates or use library documentation, but all code and reported numbers must come from your own runs. Do not use an AI assistant to generate solutions; you may use one to debug syntax errors in your own code. Note any help received in an "Acknowledgments" line at the end.